

1. Brute force approach:

Encode and Decode Strings ✓

Medium Topics Company Tags Hints

Design an algorithm to encode a list of strings to a string. The encoded string is then sent over the network and is decoded back to the original list of strings.

Machine 1 (sender) has the function:

```
String encode(List<String> strs) {  
    // ... your code  
    return encoded_string;  
}
```

Machine 2 (receiver) has the function:

```
List<String> decode(String encoded_string) {  
    // ... your code  
    return decoded_strs;  
}
```

Python Solution 1

```
1 class Solution:  
2  
3     def encode(self, strs: List[str]) -> str:  
4         return "#".join(strs)  
5  
6     def decode(self, s: str) -> List[str]:  
7         return s.split("#")  
8
```

Input:

```
strs=["Hello","World"]
```

Your Output:

```
["Hello","World"]
```

Expected output:

```
["Hello","World"]
```

Run Submit

→ I passed the test case because neither string uses # inside that string , what if the string itself contains # like for example ["ab#cd", "xyz"] In this case if we use brute force it fails the test case .

2. Optimized approach :

Accepted 51 / 51 test cases

Memory: 8 MB • Time: 28ms • Submitted at: 06/02/2026 10:02

Runtime: 28ms | Beats 99.89%

Memory: 8.0 MB | Beats 99.06%

Test Case Output: Accepted

Passed test cases: 51 / 51

Python Solution 1

```
1 class Solution:  
2     def encode(self, strs: List[str]) -> str:  
3         encode = ""  
4         for word in strs:  
5             encode += str(len(word)) + "#" + word  
6         return encode  
7     def decode(self, s: str) -> List[str]:  
8         result = []  
9         i = 0  
10        while i < len(s):  
11            j = i  
12            while s[j] != "#":  
13                j += 1  
14            length = int(s[i:j])  
15            word = s[j+1 : j+1+length]  
16            result.append(word)  
17            i = j+1+length  
18        return result
```

Run Submit

Encode and Decode Strings

Approach	Encode Time	Encode Space	Decode Time	Decode Space	Valid Solution
Brute Force (<code>"#".join()</code> + <code>split("#")</code>)	$O(n)$	$O(n)$	$O(n)$	$O(n)$	 No
Optimal (<code>length + "#" + string</code>)	$O(n)$	$O(n)$	$O(n)$	$O(n)$	 Yes

where:

`n` = total number of characters across all strings



→ ***Both method has same time and space complexity then why is the optimized method better not because of complexity because of correctness.***

Interview Answer

"Both approaches have $O(n)$ time and $O(n)$ space complexity. However, the brute-force delimiter approach is not reliable because the delimiter may appear inside the original strings. The length-prefixed encoding approach maintains the same complexity while guaranteeing correct decoding for all valid inputs."

🔥 This is one of those rare problems where the **optimized solution doesn't improve Big-O complexity—it improves correctness and robustness.**